DeskBEE: A Voice-Activated Environment Monitor and Pomodoro Desk Companion Using an ESP32 and Offline Speech Recognition

Rumman Karim
Dept. of CSE
United International University
Dhaka, Bangladesh
ID: 0112330121

Anika Islam
Dept. of CSE
United International University
Dhaka, Bangladesh
ID: 0112330512

Abdul Gofran Emon
Dept. of CSE
United International University
Dhaka, Bangladesh
ID: 0112330904

Sifat Zaman
Dept. of CSE
United International University
Dhaka, Bangladesh
ID: 0112330278

Arup Saha
Dept. of CSE
United International University
Dhaka, Bangladesh
ID: 0112330757

***Abstract*—Students and desk workers often need a simple way to structure focused work sessions and to check the conditions of the room they work in, without picking up a phone or sending their voice to a cloud service. This paper presents DeskBEE, a desk companion built around an ESP32 microcontroller that combines a Pomodoro timer with a voice-activated room environment monitor. The ESP32 drives an OLED user interface, a rotary encoder, a vibration sensor for wake-up, a buzzer, a DHT11 temperature and humidity sensor, and I2S microphone and speaker modules, while a PC running Faster-Whisper performs offline speech-to-text and returns a single command byte over a 921600-baud serial link. A planned camera-based posture warning subsystem, using an ESP32-CAM, MediaPipe and an Arduino Uno driven servo, was attempted but could not be integrated reliably, and its failure is analysed. In the final prototype, the Pomodoro timer, the OLED interface, and the spoken "room details" query with on-screen and audio feedback were demonstrated working. During testing and the demonstration every spoken query was recognised and answered; the only failures came from loose breadboard connections, not from the speech pipeline. Response time and power use were not measured.**

***Index Terms*—ESP32, Pomodoro timer, offline speech recognition, Faster-Whisper, I2S audio, DHT11, embedded systems, serial communication**

I. Introduction

A. Problem Statement and Motivation

Long study and work sessions at a desk raise two everyday problems. The first is time management: without a clear structure, it is easy to either lose focus or work for long stretches without a break. The Pomodoro Technique addresses this by splitting work into timed intervals separated by short breaks [3]. The second is awareness of the working environment: temperature and humidity affect comfort, but checking them usually means looking at a separate device or a phone, which is itself a distraction.

DeskBEE (Voice AI & Productivity Companion) was designed to address both problems with a single desk-top device. It provides a physical Pomodoro timer that can be operated with one rotary knob, and it answers a spoken question about the room by reading a sensor and replying on its display and through a speaker. The team chose this problem because it touches nearly every topic of a microprocessor and microcontroller course: GPIO and interrupt-style input, I2C displays, single-wire sensors, I2S digital audio, serial communication with a host computer, and the partitioning of work between a microcontroller and a more powerful processor.

A design goal was that speech should be processed *locally*. Commercial voice assistants typically send audio to remote servers; DeskBEE instead runs the speech recogniser on a PC on the same desk, so no audio leaves the user's own machines.

The original scope also included a camera-based posture correction feature. This feature was attempted but was not successfully integrated into the final prototype; Section V documents the attempt and the reasons it failed.

B. Related Work

Speech recognition. Whisper is a sequence-to-sequence speech recognition model trained on a large and diverse set of weakly supervised audio, and it is known for robustness to accents and background noise [4]. Faster-Whisper re-implements Whisper inference on the CTranslate2 engine and supports reduced-precision (Int8) inference on a CPU [5], which makes it practical on an ordinary laptop. On microcontrollers themselves, the usual approach is small-footprint keyword spotting, which recognises only a fixed set of words [6]. Huang *et al.* argue for moving smart-home speech recognition from the cloud to local devices to reduce energy use, latency and dependence on a single point of failure [7]; DeskBEE follows

the same local-processing idea, but places the recogniser on a nearby PC rather than on the embedded device.

Posture monitoring. MediaPipe provides a framework for real-time perception pipelines [8], and its BlazePose model estimates body landmarks from a single camera image [9]. Hsieh and Sun built PoseTrack, which uses a Raspberry Pi camera and MediaPipe Pose landmarks to detect forward lean and slouching and to give feedback through a mobile application [10]. DeskBEE's attempted posture subsystem followed a similar landmark-based approach, using an ESP32-CAM as the camera.

Productivity tools. The Pomodoro Technique [3] is widely available as phone and desktop software. A physical timer avoids having to unlock a phone, which is itself a common source of distraction.

C. Features and Contributions

The features of the final DeskBEE prototype are:

- a Pomodoro timer set and started with a single rotary encoder, with a buzzer and an on-screen "Time's Up" alert;
- an expressive OLED interface with an "Awake Face" and an automatic sleeping ("Zzz") face after 10 s without user interaction, woken again by a vibration sensor rather than a button;
- a voice query ("room", "temperature" or "details") that triggers a DHT11 reading, an OLED display of temperature and humidity, and spoken audio feedback through an I2S speaker;
- a split architecture in which a PC runs offline speech recognition with Faster-Whisper and the ESP32 acts as the real-time controller and I/O hub, linked by a minimal serial protocol.

Compared with a phone-based Pomodoro app or a cloud voice assistant, DeskBEE combines both functions in one dedicated desk device and keeps all speech processing local.

II. Proposed Method

A. Block Diagram of the Overall System

Fig. 1 shows the overall system. DeskBEE divides the work between two working tiers, plus one attempted actuator unit:

1) AI / processing layer (PC + Python). Speech-to-text is too computationally heavy for the ESP32, so it runs on a PC. A Python program receives audio from the ESP32 over serial, transcribes it offline with Faster-Whisper using the `tiny.en` model with Int8 inference on the CPU, searches the text for keywords, and sends a single command byte back.

2) Master controller (ESP32). The ESP32 owns all real-time input and output. It drives the OLED interface over I2C, reads the rotary encoder, the vibration sensor and the DHT11 sensor, captures microphone audio and plays speaker audio over I2S, sounds the buzzer, and runs the Pomodoro timer. It also decides when speech is likely to be present (using an RMS noise gate) and only then streams audio to the PC.

3) Attempted actuator unit (Arduino Uno). A separate Arduino Uno was meant to drive a servo that raised a physical posture-warning flag. It was split from the ESP32 because generating servo PWM on the ESP32 conflicted with the timing needs of the I2S audio subsystem. This unit, together with the ESP32-CAM and MediaPipe pipeline, was not successfully integrated (Section V).

The serial link is the only connection between the two working tiers, and it carries very little control traffic: one text flag and a fixed-length audio block from the ESP32, and one byte back from the PC. This keeps the two programs independent of each other's internal logic.

B. Pomodoro Timer and Desk Companion Interface

The ESP32 runs the user interface as a set of display states:

1) On start-up the OLED shows an *Awake Face*.
2) Turning the rotary encoder enters *Timer Setup* mode, and further rotation changes the selected duration between 1 and 99 minutes (the default is 25 minutes, one standard Pomodoro).
3) Pressing the encoder push-button starts the countdown, which is shown on the OLED.
4) When the countdown reaches zero, the buzzer sounds and a *Time's Up* message is displayed.
5) If there is no user interaction for the idle timeout (`IDLE_TIMEOUT`), the display switches to a *sleeping face* ("Zzz"). For testing and the demonstration the timeout was set to 10 s so that this behaviour could be shown quickly; the firmware in the project repository uses a longer timeout for everyday use. Sleeping reduces unnecessary display activity, although its effect on power use was not measured.
6) While DeskBEE is asleep, a vibration picked up by the vibration sensor (for example, a tap on the desk or the device) wakes it and returns the display to the Awake Face. The sensor output is active-low and is read on GPIO 4 with the internal pull-up enabled, so no button press is needed to wake the device. Voice queries are ignored while DeskBEE is asleep, so the user taps to wake it before asking about the room.

The timer is set and started with a single knob and its push-button, and waking the device only needs a tap, so the user does not have to look away from their work for long. The intended final behaviour is for the camera to detect whether the user is present at the desk and to put DeskBEE to sleep after 5 minutes of absence; this depends on the camera subsystem, which was not integrated (Section V), so it remains future work.

C. Voice-Activated Environment Monitor

The voice pipeline (Fig. 2) runs as follows:

1) The INMP441 I2S microphone continuously fills a 512-sample audio buffer on the ESP32 (16 kHz, 16-bit, mono).

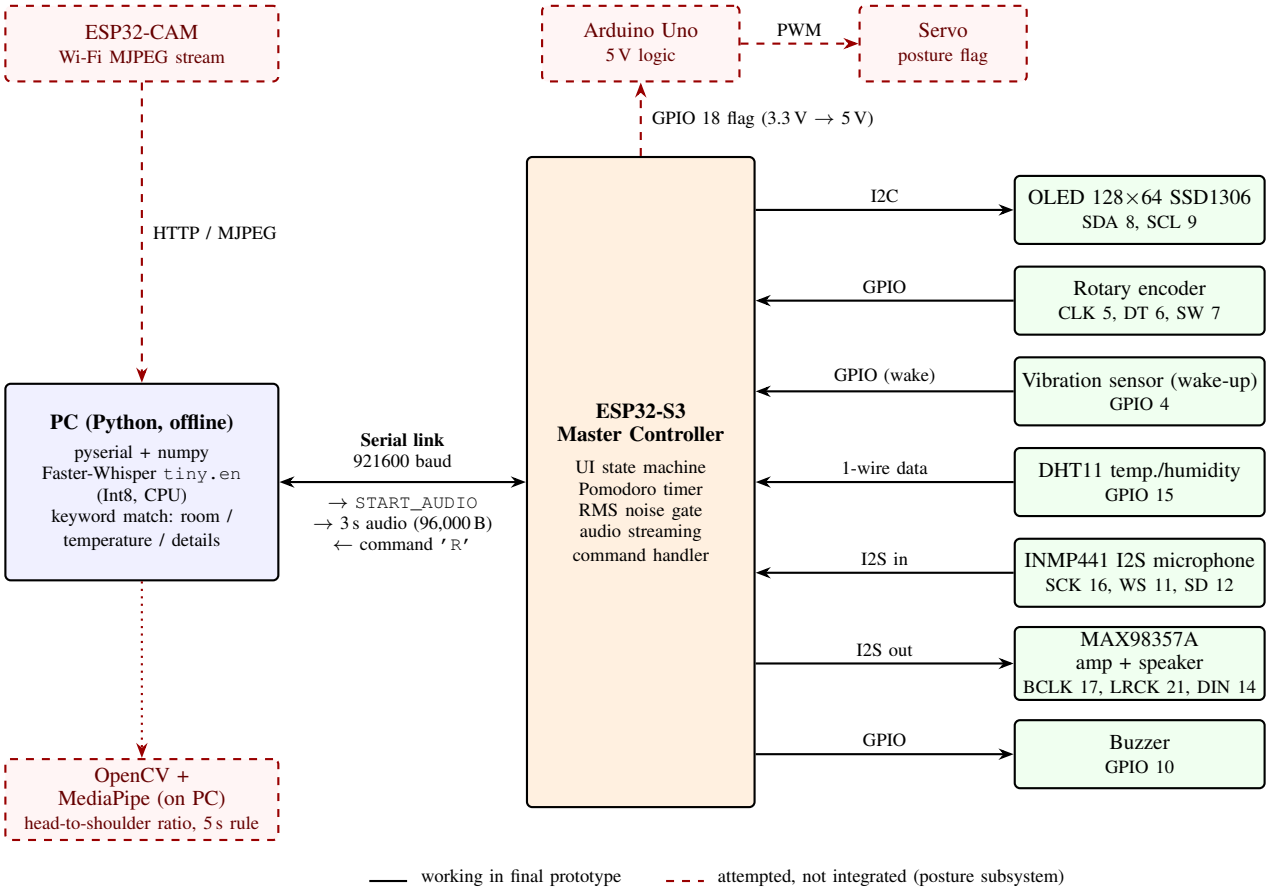


Fig. 1. Block diagram of DeskBEE. The PC performs offline speech recognition, the ESP32 is the real-time controller and I/O hub, and the dashed red blocks show the posture-warning subsystem that was attempted but not integrated into the final prototype.

2) The ESP32 computes the root-mean-square (RMS) volume of the buffer,

$$\mathrm{RMS} = \sqrt{\frac{1}{N}\sum_{n=1}^{N} x[n]^2}, \tag{1}$$

where $x[n]$ are the N samples in the buffer.
3) If the RMS value exceeds the noise gate of 1500.0, the ESP32 sends the `START_AUDIO` flag to the PC.
4) The ESP32 then streams exactly 3 s of raw audio, 96,000 bytes, over serial.
5) The Python program receives the byte stream.
6) Faster-Whisper (`tiny.en`, Int8, CPU) transcribes the audio.
7) If the transcription contains a keyword such as “room”, “temperature” or “details”, Python sends the byte `'R'` back to the ESP32.
8) On receiving `'R'`, the ESP32 reads the DHT11, shows the temperature and humidity on the OLED for 5 s, and plays a pre-compiled WAV audio array through the I2S speaker as spoken feedback. The display then returns to the screen that was showing before the query.

Several details in the Python program make recognition more dependable. The serial port is opened with a 128 KiB receive buffer so that a whole 96,000-byte block fits without loss, and a block that arrives short is reported as a buffer under-run and discarded instead of being transcribed. Faster-Whisper is run with greedy decoding (beam size 1) and its built-in voice-activity filter, which drops silence and static. It is also given the initial prompt “Show the room temperature details.”, which steers the small `tiny.en` model towards the expected vocabulary. During testing the model sometimes heard the command words as “aperture”, “better” or “boom”, so these words were added to the keyword list as a fallback.

The noise gate means that the PC only spends time on transcription when the room is loud enough for speech to be likely, and the fixed-length recording keeps the protocol simple: the PC always knows exactly how many bytes to read after `START_AUDIO`.

D. Integration and Communication Flow

Table I lists every message on the serial link. The 96,000-byte block for 3 s of audio corresponds to 32,000 bytes per second, which matches the I2S configuration of 16,000 samples/s at 2 bytes per sample. With the usual 8-N-1 framing (10 bits on the wire per byte), a 921600-baud link carries at most 92,160 bytes/s, so moving one block takes at least about 1.04 s. This is a theoretical lower bound computed from the

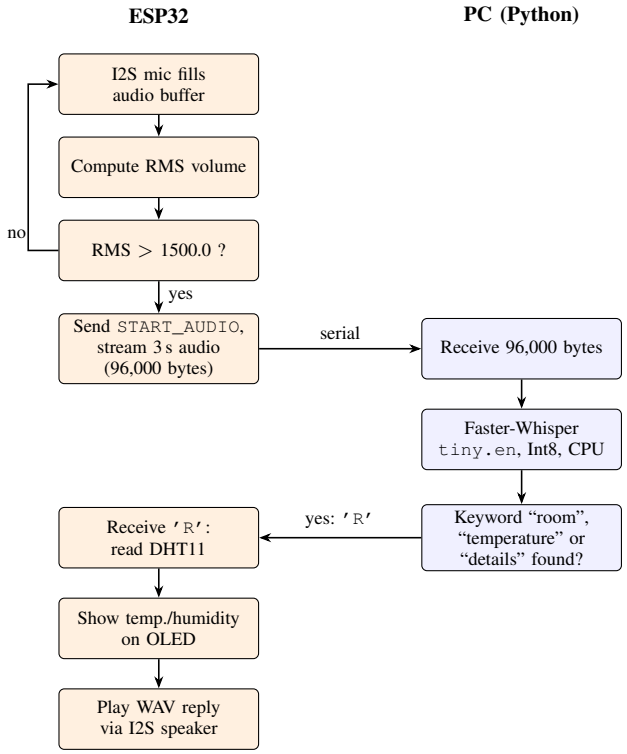


Fig. 2. Voice-activated environment monitoring pipeline. The ESP32 handles audio capture, gating and feedback; the PC handles speech recognition and keyword matching.

TABLE I
SERIAL MESSAGES BETWEEN THE ESP32 AND THE PC

Message	Direction	Meaning
`START_AUDIO`	ESP32 → PC	RMS noise gate exceeded; an audio block follows
Audio block	ESP32 → PC	3 s of raw audio, exactly 96,000 bytes
`'R'`	PC → ESP32	A room keyword was recognised; read and report the DHT11
`'1'` / `'0'`	PC → ESP32	Attempted posture subsystem: set / clear the flag signal to the Arduino Uno on GPIO 18

link settings, not a measured value; it shows why a high baud rate was chosen, since at a common rate such as 115200 baud the same block would need over 8 s.

III. IMPLEMENTED HARDWARE SYSTEM

A. List of Hardware and Software Environment

Table II lists the hardware and the ESP32 pin assignments, and Table III lists the software. The ESP32 firmware was written in C++ in the Arduino IDE. The PC program was written in Python inside a virtual environment (`.venv`).

B. Figures of the Implemented Hardware System

Fig. 3 gives a labelled overview of how the modules are arranged on the breadboard. Because the working prototype is built with dense jumper wiring that is hard to follow in a photograph, this overview was prepared as an AI-generated illustration, while the photographs in Section IV show the real hardware in operation. Fig. 4 shows the individual modules, and Fig. 5 gives the complete wiring diagram. All modules run from the ESP32-S3, which connects to the PC through its USB serial interface. The I2S microphone and the I2S amplifier use separate sets of clock and data pins, so audio capture and playback have their own signal lines.

TABLE II
HARDWARE COMPONENTS OF THE WORKING PROTOTYPE

Component	Function	ESP32 connection
ESP32-S3 development board	Master controller	–
OLED 128×64 (SSD1306) [14]	Visual UI / status	I2C: SDA GPIO 8, SCL GPIO 9
Rotary encoder with push-button (HW-040)	Timer and UI control	CLK GPIO 5, DT GPIO 6, SW GPIO 7
Vibration sensor module (SW-18010P)	Wake-up from sleep	GPIO 4 (active-low, internal pull-up)
DHT11 [15]	Temperature and humidity	GPIO 15
INMP441 I2S MEMS microphone	Voice capture	SCK GPIO 16, WS GPIO 11, SD GPIO 12
MAX98357A I2S amplifier + speaker	Audio feedback	BCLK GPIO 17, LRCK GPIO 21, DIN GPIO 14
Active buzzer	Timer alert	GPIO 10
PC / laptop	Speech recognition host	Serial link, 921600 baud
Attempted, not integrated:		
ESP32-CAM	MJPEG video for posture	Wi-Fi (to PC)
Arduino Uno + SG90 micro servo	Posture-warning flag	Flag signal on GPIO 18

TABLE III
SOFTWARE ENVIRONMENT

Software / library	Role in DeskBEE
ESP32 firmware (Arduino IDE, C++)	
`Adafruit_GFX`, `Adafruit_SSD1306`	Drawing faces, text and timer screens on the OLED
`ESP32Encoder`	Reading rotary encoder position
`DHT`	Reading temperature and humidity
`driver/i2s.h` [13]	I2S microphone capture and speaker playback
PC host (Python virtual environment)	
`faster-whisper` [5]	Offline speech-to-text, `tiny.en`, Int8 on CPU
`pyserial`	Two-way serial link with the ESP32 at 921600 baud
`numpy`	Converting the raw 16-bit audio bytes into normalised samples for Whisper
`opencv-python` [11], `mediapipe` [8]	Camera handling and pose estimation in the attempted posture subsystem

IV. RESULTS

A. Pomodoro Timer and OLED Interface

The Pomodoro subsystem was implemented and demonstrated. Fig. 6(a)–(e) shows each interface state. After power-

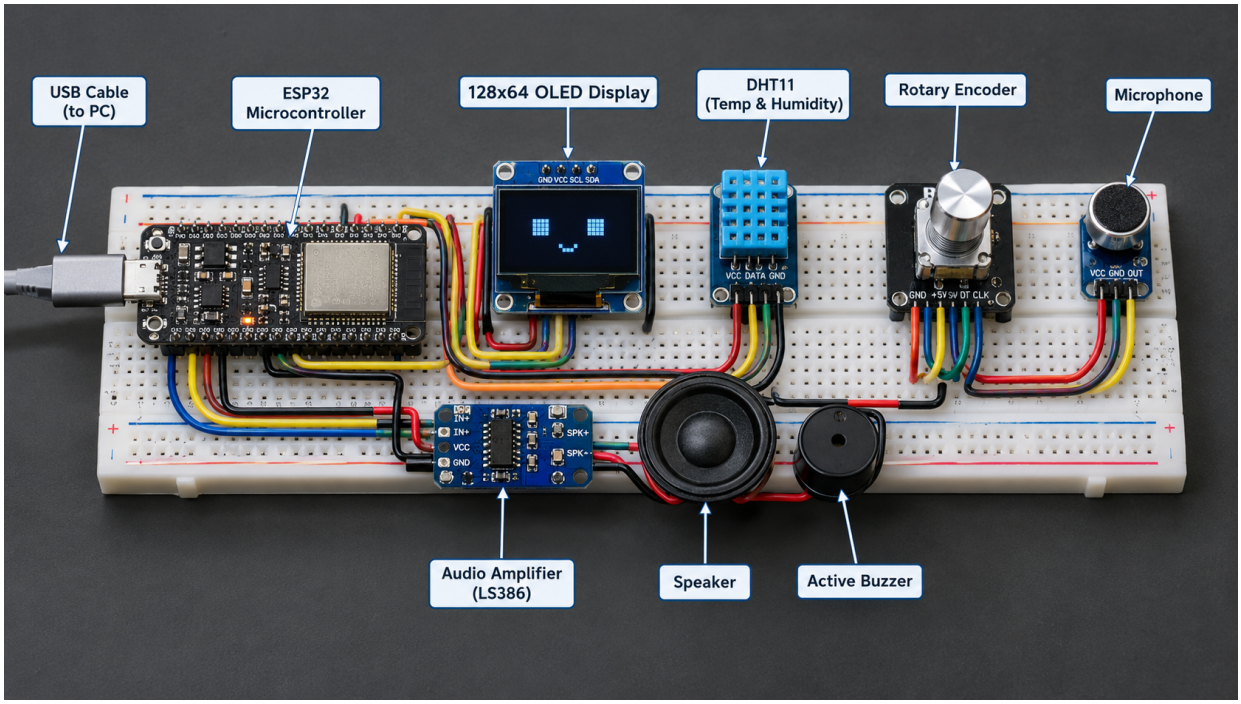
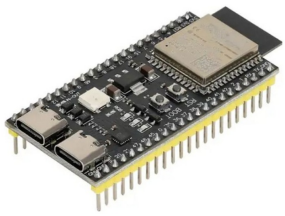


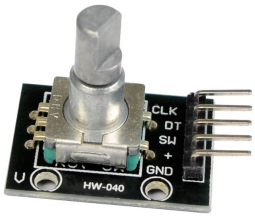
Fig. 3. Labelled overview illustration of the DeskBEE layout on a single breadboard, showing the ESP32 master controller, OLED display, DHT11 sensor, rotary encoder, microphone, audio amplifier, speaker, buzzer and the USB link to the PC. This illustration was produced with an AI image-generation tool to present the arrangement of the modules clearly; module shapes are representative (the build uses the INMP441 microphone and MAX98357A amplifier shown in Fig. 4). Photographs of the real working prototype are shown in Fig. 6.



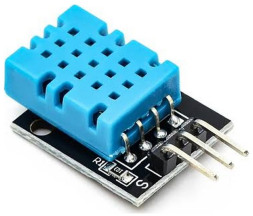
(a) ESP32-S3 dev. board



(b) OLED display



(c) Rotary encoder (HW-040)



(d) DHT11 sensor



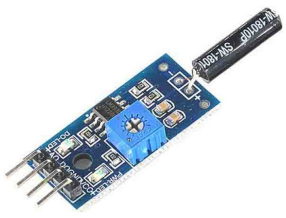
(e) INMP441 I2S microphone



(f) Speaker



(g) Active buzzer



(h) Vibration sensor (SW-18010P)

Fig. 4. Hardware modules used in DeskBEE. Representative product photographs are used for clarity; the MAX98357A amplifier board is shown in the wiring diagram (Fig. 5).

up the ESP32 displayed the Awake Face (a). Turning the encoder opened Timer Setup and changed the duration, here to 24 minutes (c); pressing the encoder started the countdown (d). When the countdown reached zero the buzzer sounded and the Time's Up message appeared (e). With the 10 s test timeout, leaving the device untouched changed the display to the sleeping face (b). A vibration detected by the vibration sensor woke the device from this state and brought back the Awake Face, without any button press.

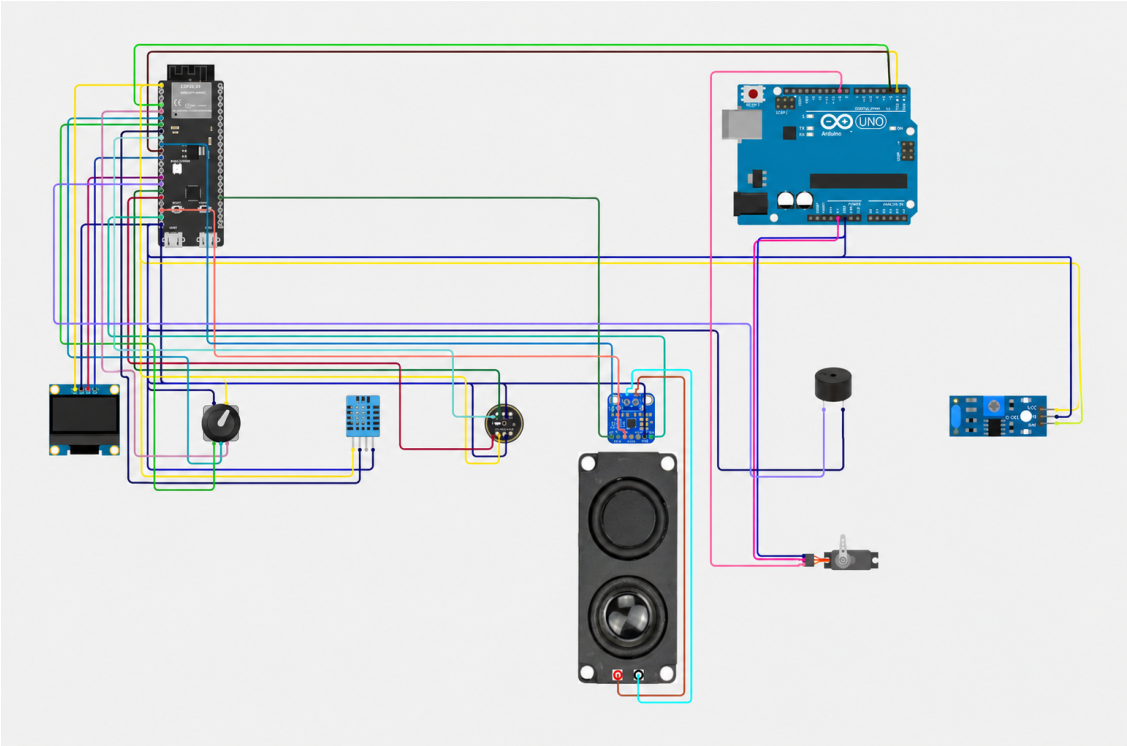


Fig. 5. Complete wiring diagram of DeskBEE, prepared in an online circuit-diagram tool. The ESP32-S3 (top left) connects to the OLED, rotary encoder, DHT11, INMP441 microphone, MAX98357A amplifier with speaker, buzzer and vibration sensor; pin assignments follow Table II. The Arduino Uno and servo of the attempted posture-warning subsystem are also shown (top and bottom right).

B. Voice-Activated Environment Monitor

The voice-activated environment monitor was implemented and demonstrated. Fig. 7 shows one complete query as logged by the Python program. After the model loaded, the program listened on the ESP32's serial port. When the user spoke, the RMS value rose above the 1500.0 gate, the ESP32 sent `START_AUDIO`, and the full 96,000-byte buffer was received. Faster-Whisper transcribed it as "show the details.", which contains a command keyword, so the program sent `'R'` to the ESP32. The ESP32 then read the DHT11 and displayed the result on the OLED, 32.3 °C and 58.0 % relative humidity in Fig. 6(f), and played the stored WAV reply through the I2S speaker.

While the Python program is running it holds the ESP32's serial port, so the Arduino IDE serial monitor cannot be opened at the same time. For this reason the Python console in Fig. 7 is the record of the serial exchange: each of its lines corresponds to one stage of the protocol in Table I.

This result shows the intended division of labour working end to end: the PC handled the heavy speech recognition, while the ESP32 handled real-time audio capture, sensing, display and audio playback.

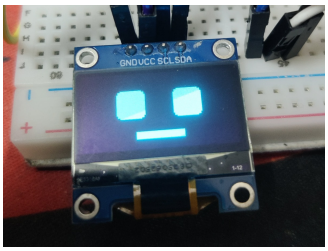
Throughout testing and the final demonstration, every spoken room query was recognised and answered correctly. The only failures occurred when a jumper wire on the breadboard had worked loose, which interrupted the microphone or sensor connection; once the connection was restored the pipeline worked again. These failures were caused by the prototype wiring, not by the speech-recognition pipeline. End-to-end response time and false-trigger rate were not formally measured.

TABLE IV
STATUS OF DESKBEE FEATURES IN THE FINAL PROTOTYPE

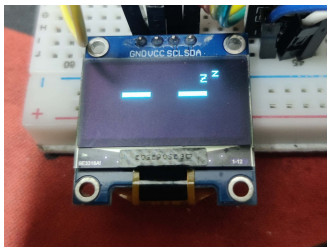
Feature	Status
Pomodoro timer (encoder, buzzer, Time's Up)	Implemented, demonstrated
OLED interface (awake / sleeping faces)	Implemented, demonstrated
Vibration-sensor wake-up	Implemented, demonstrated
DHT11 sensing in voice workflow	Implemented, demonstrated
Faster-Whisper speech processing on PC	Implemented, demonstrated
I2S audio capture and WAV playback	Implemented, demonstrated
ESP32-CAM posture video pipeline	Attempted, not integrated
MediaPipe posture detection	Worked standalone, not integrated
Arduino Uno + servo posture flag	Attempted, not integrated
Posture warning as a final feature	Not functional

C. Summary of Feature Status

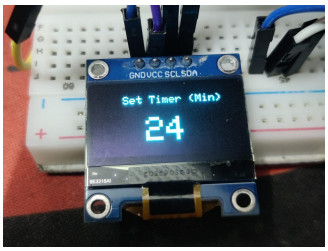
Table IV summarises which features work in the final prototype. The prototype should be judged on the Pomodoro



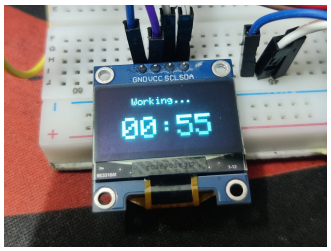
(a) Awake face



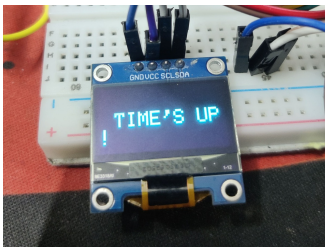
(b) Sleeping face (Zzz)



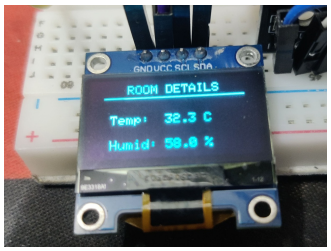
(c) Timer setup



(d) Timer counting down



(e) Time's Up alert



(f) Room details after voice query

Fig. 6. Photographs of the OLED interface on the working prototype: (a) Awake Face, (b) sleeping face, (c) Timer Setup at 24 minutes, (d) countdown, (e) Time's Up alert, and (f) room details after a voice query (32.3 °C, 58.0 % relative humidity).

and voice-controlled environment-monitoring functions; the posture subsystem is discussed separately below.

V. Posture Detection Attempt and Failure Analysis

A. Intended Design

The posture-warning subsystem (dashed blocks in Fig. 1; hardware in Fig. 8) was planned as follows. An ESP32-CAM would stream MJPEG video over Wi-Fi. The Python program would fetch the stream with OpenCV and run MediaPipe pose estimation [8], [9] to compute a head-to-shoulder ratio for the user. If the ratio indicated bad posture continuously for 5 s, Python would send `'1'` to the ESP32 (and `'0'` once posture recovered). The ESP32 would then raise or clear a flag signal on GPIO 18 to an Arduino Uno, and the Uno would drive a servo to raise a physical warning flag.

The PC-side detection itself worked when tested on its own. Fig. 8(b) and (c) show the detector window: the program stores a baseline head ratio for the user's upright posture (0.82), compares each frame against it, labels the posture as good or bad, and runs the 5 s timer while bad posture persists. The failure occurred when this detector had to be combined with the rest of the system, as described next.

B. Reasons for Failure

1) Timer conflict on the ESP32. Driving the servo directly from the ESP32 caused a hardware timer / PWM conflict with the timing-sensitive I2S audio subsystem. In the attempted design, servo PWM and I2S audio could not both be kept working reliably at the same time.

2) Decoupling the actuator. To remove the conflict, the servo was moved to an Arduino Uno. This is a reasonable design response: it takes the actuator's timing requirements off the ESP32, which is already busy with audio.

3) A longer chain of nodes. The result was a three-node chain, PC → ESP32 → Arduino Uno / servo. Each extra link is another interface that must work, and the chain introduced practical communication problems.

4) Logic-level mismatch. The ESP32 uses 3.3 V logic while the Arduino Uno uses 5 V logic, which added electrical and interface complexity to the signal between the two boards.

5) Serial and processing contention. The Python host had to handle the high-rate audio serial stream and the HTTP/MJPEG video stream from the ESP32-CAM at the same time. This led to serial buffer overruns and timing problems when both ran together.

C. Final Status

The posture-detection and servo-warning subsystem was not successfully integrated, and posture warning is not a feature of the final prototype. The attempt was still useful: it exposed the concrete limits of the architecture, namely shared timer resources on the ESP32, the cost of each extra node and voltage domain, and the bandwidth limits of a single host handling audio and video together.

VI. Discussion

A. Limitations

The following cases show where the current prototype does not work as intended:

- *Noisy rooms.* The trigger is an RMS volume threshold, not speech detection. Any loud sound above 1500.0, such as a door closing or music, can start a recording and transcription, and a quiet speaker may not cross the gate at all.
- *Fixed 3 s window.* A request that starts late or lasts longer than 3 s may be cut off, and the keywords may then be missed.
- *Keyword matching.* Only transcripts containing one of the keywords trigger a response. A differently worded request gives no response, and the fallback words added for common mishearings ("aperture", "better", "boom") could also trigger a response from unrelated speech.
- *PC dependency.* Voice features need the PC to be on, connected and running the Python program; without it only the Pomodoro timer works.

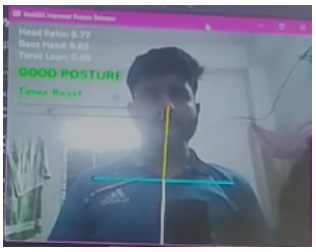
```
(.venv) PS F:\Study\University\9th Trimester\CSE 4326 - Microprocessor and Microcontroller Laboratory\microLab\py_code>python .\voice_pipeline.py
Loading Whisper AI Model (Int8 / CPU)...
Model Loaded!
Listening to COM5 for ESP32 microphone data...

[!] ESP32 detected speech! Downloading 3-second audio buffer...
Buffer received. Transcribing...
Heard: 'show the details.'
>>> COMMAND MATCH! Sending 'R' to ESP32 screen... <<<
```

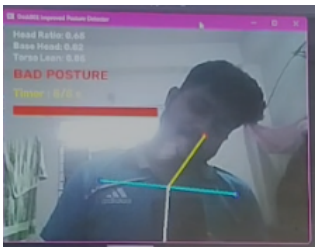
Fig. 7. Python host console during a voice query. The ESP32 detects speech, the 3 s buffer is received and transcribed offline by Faster-Whisper (heard: "show the details."), and a matching keyword causes the `'R'` command to be sent to the ESP32.



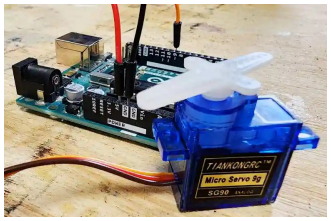
(a) ESP32-CAM module



(b) Good posture (head ratio 0.79)



(c) Bad posture (head ratio 0.65)



(d) Arduino Uno and SG90 servo

Fig. 8. The attempted posture-warning subsystem. (b) and (c) are photographs of the PC-side detector during standalone testing: with a calibrated baseline head ratio of 0.82, a ratio of 0.79 is classed as good posture, while 0.65 is classed as bad posture and starts the 5 s warning timer (red bar). (a) and (d) are representative images of the modules used. The subsystem was not integrated into the final prototype.

- *Vibration wake-up.* Because any vibration above the sensor's threshold wakes the device, an accidental bump of the desk may wake it when it is not needed, and a very light tap may not wake it.
- *Breadboard wiring.* The prototype is built on a breadboard with jumper wires, and a loose connection was the only cause of failed queries during testing. A soldered board or PCB would be needed for long-term use.
- *Sensor precision.* The DHT11 has limited range and accuracy according to its datasheet [15], so readings indicate room conditions only roughly.
- *Posture warning.* This feature is not functional (Section V).

B. Conflicting Requirements and Trade-offs (P2)

The main conflict was between real-time audio and actuator control. The I2S audio subsystem needs steady, uninterrupted timing, while the servo needs its own PWM timing. Putting both on the ESP32 saved a board and a link but broke reliability; moving the servo to an Arduino Uno solved the timer conflict but added a third node, a second voltage domain and more communication to keep in sync. A similar trade-off was made for speech recognition: running Faster-Whisper on a PC gives much more capable recognition than the ESP32 could run on its own, at the cost of requiring a PC and a serial link.

C. Alternative Designs (P3)

On-device keyword spotting. A small keyword-spotting model on the microcontroller [6] would remove the need for a PC and the 96,000-byte transfer. It would make the device standalone, but it can recognise only a small fixed vocabulary and would compete with the existing firmware for memory and processing time.

A dedicated PWM driver or a single-board computer. Instead of an Arduino Uno, the servo could be driven by a dedicated PWM driver module controlled over the existing I2C bus, which would avoid a second microcontroller and a 5 V logic signal; whether this fully removes the timing problem would need to be tested. Alternatively, a single-board computer with a camera, as in PoseTrack [10], could run pose estimation next to the camera and drive the servo directly, so the PC would no longer have to handle audio and video at once. This would cost more but remove the network video stream.

Level shifting. Whatever actuator node is used, a proper logic-level shifter between 3.3 V and 5 V signals would remove one source of the interface problems seen in the posture attempt.

D. Impact (P4)

DeskBEE addresses a problem every CSE student faces: managing long study sessions at a desk. Building it also required work outside core computer science, including digital audio (sampling, RMS level detection and I2S), environmental sensing, and, for the posture attempt, basic ergonomics and human pose estimation.

E. Standards, Manuals and Ethics (P5)

The project relied on interface standards and vendor documentation: the I2S bus specification [12] and Espressif's I2S driver documentation [13], the SSD1306 controller datasheet for the I2C display [14], and the DHT11 datasheet [15]. Using these in practice went beyond the documents in two places: the resource conflict between I2S and servo PWM on the ESP32, and the 3.3 V/5 V logic mismatch between the ESP32 and the Arduino Uno. On the ethical side, the microphone and the planned camera both collect personal data. DeskBEE

TABLE V
COMPLEX ENGINEERING PROBLEM MAPPING

P1	P2	P3	P4	P5	P6	P7
✓	✓	✓	✓	✓		✓

P1 – Depth of Knowledge, P2 – Conflicting Requirements, P3 – Depth of Analysis, P4 – Familiarity of Issues, P5 – Extent of Applicable Codes, P6 – Extent of Stakeholders, P7 – Interdependence.

keeps speech recognition on the user's own PC and does not send audio to a cloud service, and the microphone only records after the noise gate is exceeded. For transparency, the overview illustration in Fig. 3 is identified as AI-generated, and the module images in Figs. 4 and 8(a, d) are identified as representative product photographs; all OLED and console results are from the team's own prototype.

F. Module Decoupling and Interdependence (P7)

DeskBEE's modules are deliberately decoupled. The PC and the ESP32 share only the three serial messages in Table I, so the speech model or keyword list can change without touching the firmware, and the Pomodoro timer works with no PC at all. However, the voice feature does depend on hardware and software spread across two devices: the ESP32's microphone, gate and speaker, and the PC's recogniser. The posture attempt showed the limit of this approach: each extra decoupled node removed one hardware conflict but added another interface that had to work reliably.

VII. CONCLUSION

DeskBEE combines a Pomodoro timer with a voice-activated room environment monitor on an ESP32, using a PC running Faster-Whisper for offline speech recognition. In the final prototype, the timer, the OLED interface, and spoken room-condition queries with display and audio feedback were demonstrated working, and every spoken query in testing was answered except when a breadboard wire had come loose; response time and power use have not yet been measured. The planned posture warning subsystem was not integrated, because of an I2S/PWM timer conflict on the ESP32, a 3.3 V/5 V logic mismatch, and contention between audio and video streams on the host. Future work includes using the camera to detect the user's presence and put DeskBEE to sleep after 5 minutes of absence, moving the circuit from the breadboard to a soldered board, measuring response time, replacing the RMS gate with proper voice-activity detection, testing on-device keyword spotting so the device can work without a PC, and redesigning the posture subsystem around a dedicated PWM driver or a single-board computer with proper level shifting. The project source code and a demonstration video are available online [1], [2].

ACKNOWLEDGMENT

The authors thank Md. Shafqat Talukder, Lecturer, Department of CSE, United International University, for guidance throughout the CSE 4326 Microprocessor and Microcontroller Laboratory course. An AI image-generation tool was used to prepare the overview illustration in Fig. 3; the wiring diagram was drawn with an online circuit-diagram tool.

REFERENCES

[1] R. Karim, A. Islam, A. G. Emon, S. Zaman, and A. Saha, "DeskBEE lab project," GitHub repository. [Online]. Available: https://github.com/rumman999/deskbee-lab-project

[2] DeskBEE project demonstration video, Google Drive. [Online]. Available: https://drive.google.com/drive/folders/1tZC6TZ2J6uB5XZFCrDfc70PA-nQj4Uqx?usp=sharing

[3] F. Cirillo, *The Pomodoro Technique: The Life-Changing Time-Management System*. London, U.K.: Virgin Books, 2018.

[4] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proc. 40th Int. Conf. Machine Learning (ICML)*, 2023, pp. 28492–28518.

[5] SYSTRAN, "faster-whisper: Faster Whisper transcription with CTranslate2," GitHub repository. [Online]. Available: https://github.com/SYSTRAN/faster-whisper

[6] I. López-Espejo, Z.-H. Tan, J. H. L. Hansen, and J. Jensen, "Deep spoken keyword spotting: An overview," *IEEE Access*, vol. 10, pp. 4169–4199, 2022.

[7] P. Huang, I. Ullah, X. Wei, T. A. Ahanger, N. Hassan, and Z. H. Shah, "Towards energy-efficient and low-latency voice-controlled smart homes: A proposal for offline speech recognition and IoT integration," arXiv:2506.07494, 2025.

[8] C. Lugaresi *et al.*, "MediaPipe: A framework for building perception pipelines," arXiv:1906.08172, 2019.

[9] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, and M. Grundmann, "BlazePose: On-device real-time body pose tracking," arXiv:2006.10204, 2020.

[10] Y.-C. Hsieh and Y. Sun, "An intelligent mobile application to monitor and correct sitting posture using Raspberry Pi and MediaPipe pose detection," arXiv:2508.11683, 2025.

[11] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, 2000.

[12] Philips Semiconductors, "I^2S bus specification," 1986 (rev. 1996).

[13] Espressif Systems, "Inter-IC Sound (I2S)," ESP-IDF Programming Guide. [Online]. Available: https://docs.espressif.com/projects/esp-idf/

[14] Solomon Systech, "SSD1306: 128 x 64 dot matrix OLED/PLED segment/common driver with controller," datasheet.

[15] Aosong Electronics, "DHT11 temperature and humidity sensor," datasheet.